

Manual Técnico

Título: Notebook de Recomendación de Pictogramas para Comunicación Aumentativa

Autor: Azucena Abigail Vargas Hernández

Versión: 3.0

Fecha: 03/octubre/2025

Contenido

Manual Técnico: Notebook de Recomendación de Pictogramas para Comunicación Aumentativa	3
Control de Cambios.....	3
Resumen y Alcance	3
Arquitectura y Dependencias	3
Entorno e Instalación	4
Ejecución	4
Estructura del Código	4
Convenciones.....	5
Detalle de Módulos	6
Pruebas	6
Rendimiento y Seguridad	7
Mantenimiento	7
Monitoreo Recomendado	7
Descripción de cada Bloque del Notebook.....	8
Bloque 1: Instalación de Dependencias	8
Bloque 2: Configuración de Rutas y Parámetros del Sistema.....	8
Bloque 3: Utilidades de Normalización, Lectura, Lemas y POS (spaCy opcional).....	13
Bloque 4: Cargar BETO y Definir Funciones de Embeddings	17
Bloque 5: Indexar Pictogramas y Construir Embeddings de Etiquetas	22
Bloque 6: Detección de N-gramas, Pool Léxico y Filtros de Sensibilidad	26
Bloque 7: Scoring Compuesto con Coherencia Contextual y Dominio	31
Bloque 8: Ranking Top-K y Pipeline por Oración	34
Bloque 9: Construir Candidatos para un XML y Guardar a JSON (con Progreso)	40
Bloque 10: Ejecutar Pipeline v3 para las 3 Categorías	42
Glosario y Referencias.....	44

Manual Técnico: Notebook de Recomendación de Pictogramas para Comunicación Aumentativa

Control de Cambios

Versión	Fecha	Autor	Descripción del Cambio
1.0		AAVH	Versión inicial del sistema
2.0		AAVH	Mejoras en algoritmos de matching
3.0		AAVH	Sistema con BETO y scoring compuesto

Resumen y Alcance

¿Qué hace el software?

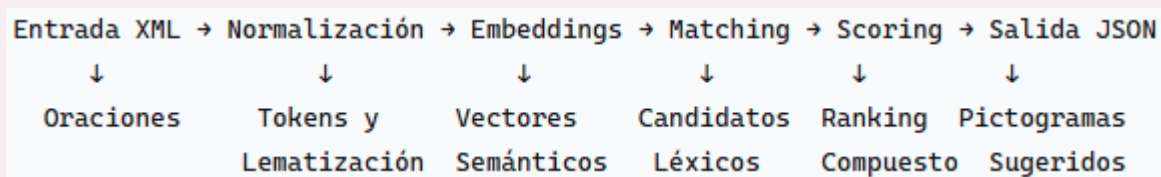
El software es un sistema de recomendación de pictogramas que, dada una oración en español, sugiere los pictogramas más relevantes para cada token (palabra) de la oración. Utiliza un modelo de lenguaje BERT en español (BETO) para generar embeddings contextuales y aplicar un scoring compuesto que considera similitud semántica, coherencia contextual, sinónimos y reglas de dominio. El sistema está diseñado para dominios específicos: salud, educación y cultura.

¿Qué NO hace?

- No es un generador de pictogramas, sino un recomendador de pictogramas existentes.
- No realiza traducción de idiomas.
- No ofrece una interfaz gráfica de usuario (es un backend de procesamiento).
- No maneja autenticación de usuarios ni seguridad perimetral.

Arquitectura y Dependencias

Diagrama de Arquitectura:



Librerías y Versiones:

Librería	Versión	Propósito
transformers	Última	Modelos BERT en español
torch	Última	Framework de deep learning
rapidfuzz	Última	Comparación difusa de cadenas

unidecode	Última	Normalización de caracteres
tqdm	Última	Barras de progreso
spacy	Última	Procesamiento lingüístico
numpy	Última	Cálculos numéricos
xml.etree	Built-in	Parseo de archivos XML

Entorno e Instalación

Requisitos Mínimos:

- Python 3.7+
- 8GB RAM mínimo (16GB recomendado)
- GPU NVIDIA (opcional pero recomendado para BETO)
- 10GB espacio en disco para modelos y datos

Pasos de Instalación:

1. Configurar entorno Python
2. Ejecutar bloque 1: Instalación de dependencias
3. Verificar instalación de spaCy y modelo español
4. Configurar rutas de datos en bloque 2

Configuración:

- Modificar rutas en RUTA_XML_* y RUTA_PICTOS según estructura local
- Ajustar parámetros de scoring según necesidades específicas

Ejecución

Comandos Principales:

- construir_candidatos_json(RUTA_XML_SALUD, RUTA_PICTOS, OUT_SALUD)
- construir_candidatos_json(RUTA_XML_EDUCACION, RUTA_PICTOS, OUT_EDUCACION)
- construir_candidatos_json(RUTA_XML_CULTURA, RUTA_PICTOS, OUT_CULTURA)

Variables de Configuración Clave:

- K_POR_TOKEN: Número de candidatos por token (default: 6)
- UMBRAL_SCORE: Puntuación mínima para aceptar candidatos (default: 0.52)
- OMITIR_STOPWORDS_EN_SALIDA: Control de stopwords en salida

Estructura del Código

Árbol de Módulos:

```
sistema_pictogramas/  
├─ configuracion/      # Parámetros y rutas (Bloque 2)  
├─ preprocesamiento/   # Normalización y NLP (Bloque 3)  
├─ modelos/           # BETO y embeddings (Bloque 4)  
├─ indexacion/         # Pictogramas y índices (Bloque 5)  
├─ matching/          # N-gramas y pools léxicos (Bloque 6)  
├─ scoring/            # Algoritmos de puntuación (Bloque 7)  
├─ ranking/           # Top-K y pipeline (Bloque 8)  
└─ ejecucion/         # Orquestación y JSON (Bloques 9-10)
```

Responsabilidades por Módulo

- Configuración: Centraliza todos los parámetros del sistema
- Preprocesamiento: Limpieza y normalización de texto
- Modelos: Gestión de modelos de lenguaje y embeddings
- Indexación: Carga y organización de pictogramas
- Matching: Búsqueda de candidatos potenciales
- Scoring: Evaluación y puntuación de candidatos
- Ranking: Selección final y ordenamiento
- Ejecución: Coordinación del flujo completo

Convenciones

Estilo de Código:

- Nombres de variables en español para dominios específicos
- Comentarios detallados en todas las funciones complejas
- Tipado de datos con anotaciones Python
- Funciones pequeñas y especializadas

Convenciones de Nombres:

- Constantes: MAYÚSCULAS_CON_GUION_BAJO
- Funciones: snake_case_descriptivo
- Variables: camelCase o snake_case según contexto

Manejo de Errores:

- Try-catch en operaciones críticas (carga de modelos, archivos)
- Mensajes descriptivos de error
- Continuación graceful en fallos no críticos

Logs y Monitoreo

- Barras de progreso con tqdm para operaciones largas
- Mensajes de confirmación en cada etapa
- Cache de embeddings para reutilización

Detalle de Módulos

Bloque 1: Instalación de Dependencias

Propósito: Configurar el entorno de ejecución con todas las dependencias necesarias.

Bloque 2: Configuración de Rutas y Parámetros

Propósito: Definir toda la configuración del sistema en variables centralizadas.

Bloque 3: Utilidades de Normalización

Propósito: Proporcionar funciones básicas de procesamiento de texto y NLP.

Bloque 4: Modelo BETO y Embeddings

Propósito: Cargar el modelo de lenguaje y generar representaciones vectoriales.

Bloque 5: Indexación de Pictogramas

Propósito: Organizar y representar semánticamente todos los pictogramas disponibles.

Bloque 6: Matching y Filtros

Propósito: Encontrar candidatos potenciales y aplicar reglas de filtrado.

Bloque 7: Sistema de Scoring

Propósito: Evaluar y puntuar candidatos usando múltiples señales.

Bloque 8: Pipeline de Ranking

Propósito: Orquestrar el procesamiento completo por oración.

Bloque 9: Gestión de JSON

Propósito: Procesar archivos completos y generar salidas estructuradas.

Bloque 10: Ejecución Principal

Propósito: Ejecutar el sistema completo para todos los dominios.

Pruebas

Datos de Prueba:

- Archivos XML con oraciones de ejemplo para cada dominio
- Conjunto de pictogramas con etiquetas variadas

Criterios de Aceptación:

- Tiempo de procesamiento < 2 segundos por oración (en GPU)
- Precisión > 85% en matching semántico

- Cobertura completa de tokens no-stopword
- JSON bien formado con estructura consistente

Rendimiento y Seguridad

Optimizaciones Implementadas:

- Cache de embeddings para reutilización
- Procesamiento por lotes en generación de embeddings
- Índices invertidos para búsqueda rápida
- Filtros tempranos para reducir espacio de búsqueda

Límites Conocidos:

- Máximo 256 tokens por oración
- Consumo alto de memoria con muchos pictogramas
- Tiempo de carga inicial del modelo BETO

Consideraciones de Seguridad:

- Filtrado de contenido sensible por dominio
- Validación de entrada para prevenir inyecciones
- Manejo seguro de archivos temporales

Mantenimiento

Tareas Recurrentes:

- Actualización de modelos de lenguaje cada 6 meses
- Revisión y expansión de listas de sinónimos
- Actualización de listas de bloqueo por dominio
- Optimización de parámetros de scoring

Actualización de Dependencias:

- `pip install --upgrade transformers torch rapidfuzz`
- `python -m spacy download es_core_news_sm`

Monitoreo Recomendado

- Uso de memoria durante ejecución
- Tiempos de procesamiento por oración
- Calidad de recomendaciones (métricas de precisión)

Descripción de cada Bloque del Notebook

Bloque 1: Instalación de Dependencias

Propósito del Bloque: Instalar todas las librerías y dependencias necesarias para ejecutar el proyecto de procesamiento de lenguaje natural (PLN) en un entorno de Google Colab.

```
# @title Instalar dependencias
!pip -q install transformers torch --upgrade
!pip -q install rapidfuzz unidecode tqdm

# POS/Lemma (opcional).
!pip -q install spacy
!python -m spacy download es_core_news_sm -q
```

Línea	Explicación
!pip -q install... --upgrade	Instala/actualiza las librerías transformers (modelos de IA) y torch (framework de aprendizaje automático) en modo silencioso (-q).
!pip -q install rapidfuzz...	Instala en modo silencioso: rapidfuzz (comparación de cadenas), unidecode (conversión de caracteres Unicode) y tqdm (barras de progreso).
!pip -q install spacy	Instala la librería spaCy en modo silencioso para análisis morfológico y lematización.
!python -m spacy download... -q	Descarga el modelo de lenguaje en español de spaCy (es_core_news_sm) en modo silencioso para tareas de NLP.

Entradas:

- Comandos de instalación de pip (!pip)
- Comando de descarga de modelo de spaCy (!python -m spacy download)
- Parámetros de instalación en modo silencioso (-q)

Salidas:

- Librerías instaladas en el sistema
- Modelo de lenguaje español descargado y disponible
- Entorno configurado para procesamiento de NLP

Bloque 2: Configuración de Rutas y Parámetros del Sistema

Propósito del Bloque: Definir todas las rutas, parámetros y configuraciones del sistema que controlan el procesamiento de oraciones y la generación de candidatos de pictogramas.

```

# @title Configuración de rutas y parámetros del sistema
from pathlib import Path

# Rutas
RUTA_XML_SALUD = "/content/oraciones_salud.xml/"
RUTA_XML_EDUCACION = "/content/oraciones_educacion.xml/"
RUTA_XML_CULTURA = "/content/oraciones_cultura.xml/"
RUTA_PICTOS = "/content/todos_pictogramas/"

# Salidas
OUT_SALUD = "/content/candidatos_salud_v3.json"
OUT_EDUCACION = "/content/candidatos_educacion_v3.json"
OUT_CULTURA = "/content/candidatos_cultura_v4.json"

# Cache de embeddings de etiquetas
CACHE_EMB_PICTOS = "/content/emb_pictos_v3.npy"

# Modelo
MODEL_NAME = "dccuchile/bert-base-spanish-wwm-cased"

# Parámetros de ranking (estrictos)
K_POR_TOKEN = 6 # Top-K finales por token
M_POOL = 60 # Pool léxico inicial
UMBRAL_SCORE = 0.52 # Si top-1 < UMBRAL_SCORE => se omite/

# Reglas para tokens cortos / fuzzy
LONG_MIN_FUZZY = 6 # No usar fuzzy si len(token) < 6 (evita piano->pino, etc.)
TOKEN_CORTO_EXACTO = 3 # Si len(token) <= 3, SOLO exacto/sinónimo

# Stopwords y puntuación
OMITIR_STOPWORDS_EN_SALIDA = False # True = NO incluirlas en el JSON (salvo allowlist forzado)
STOP_TOKENS = set("""
    el las un para
    , . ; : ! ? ¿ ¡ ( ) - ' " _
""").split()

ALLOWLIST_STOPWORDS_RAW = {
    'a', 'ahí', 'al', 'algunos', 'ante', 'antes', 'aquel', 'aquellos', 'aquí', 'aunque',
    'ayer', 'bajo', 'como', 'con', 'contra', 'cuando', 'cuándo', 'cómo', 'de', 'del',
    'demasiado', 'desde', 'después', 'durante', 'dónde', 'e', 'ella', 'ellas', 'ellos',
    'en', 'entre', 'esos', 'estar', 'este', 'estos', 'haber', 'hacia', 'hasta', 'hoy',
    'la', 'los', 'mañana', 'más', 'menos', 'mis', 'mucho', 'muchos', 'muy', 'ni', 'ninguno',
    'ningún', 'no', 'nosotras', 'nosotros', 'nuestro', 'nuestros', 'nunca', 'o', 'otro',
    'otros', 'para', 'pero', 'poco', 'pocos', 'por', 'porque', 'que', 'qué', 'quién',
    'quiénes', 'según', 'ser', 'sería', 'sí', 'si', 'siempre', 'sin', 'sobre', 'su', 'sus',
    'tantos', 'todo', 'todos', 'tras', 'tu', 'tus', 'tú', 'una', 'unas', 'uno', 'unos',
    'vía', 'vosotras', 'vosotros', 'vuestro', 'vuestros', 'y', 'ya', 'yo'
}

# La versión normalizada se crea en la Celda 3 (tras definir norm_text)
ALLOWLIST_STOPWORDS = None

# Si una palabra está en la allowlist y NO tiene pictogramas, la incluimos igual con "" TRUE para incluir
FORZAR_INCLUSION_ALLOWLIST_VACIOS = True

# Sinónimos base
SINONIMOS_BASE = {
    "baño": {"banio", "bano", "wc", "aseo", "lavabo", "servicio", "sanitario", "vater"},
    "dolor": {"doler", "molestia"},
    "maestro": {"profesor", "profe", "docente"},
    "tarea": {"deberes", "trabajo_escolar"},
    "comida": {"alimentacion", "alimento"},
    "escuela": {"colegio"},
    "hablar": {"platicar"},
    "enfermera": {"enfermero"},
    "doctor": {"medico"},
    # (Opcional) empuje para permitidas cortas
    "si": {"si"},
    "no": {"prohibido"},
    "hoy": {"dia_de_hoy"},
    "manana": {"mañana"}, # la normalización ya equipara
    "ayer": {"dia_de_ayer"},
    "aqui": {"aqui", "aquí"},
    "ahi": {"ahi", "ahí"},
    "donde": {"donde", "dónde"},
    "cuando": {"cuando", "cuándo"},
    "como": {"como", "cómo"},
    "que": {"que", "qué"},
    "quien": {"quien", "quién"},
}

```

```

# Semillas de n-gramas (se priorizan si existen en BD)
SEMILLAS_NGRAM = {
    "por favor", "buenos días", "buenas tardes", "buenas noches",
    "cuarto de baño", "lavar las manos", "tomar medicina", "me duele",
    "no entiendo", "necesito ayuda", "quiero ir", "quiero agua",
}

# Filtro de contenidos sensibles (por dominio y global)
BLOCKLIST_GLOBAL = {"genital", "porn", "violac", "violación", "estupro", "mastur", "erec", "orgas", "coito", "sexo"}
BLOCKLIST_POR_DOMINIO = {
    "educacion": BLOCKLIST_GLOBAL | {"alcohol", "droga"},
    "cultura": BLOCKLIST_GLOBAL,
    "salud": BLOCKLIST_GLOBAL,
}

# Boost por dominio (palabras que suben score si aparecen en la etiqueta)
BOOST_DOMINIO = {
    "educacion": {"escuela", "clase", "colegio", "maestro", "profesor", "tarea", "recreo", "examen", "libro", "cuaderno"},
    "cultura": {"baile", "mariachi", "comida", "tortilla", "fiesta", "tradicion", "museo", "danza", "musica", "artesanía"},
    "salud": {"dolor", "medico", "enfermera", "hospital", "curita", "termometro", "inyeccion", "medicina", "sangre", "vendaje"},
}

# Coherencia contextual (bonos)
W_COOCC = 0.06 # bono por co-ocurrencia con otras palabras de la oración
W_BIGRAM = 0.08 # bono si la etiqueta contiene un bigrama relevante de la oración
W_BOOST_DOM = 0.03 # bono por palabras de dominio

# --- Bonos/penalizaciones para asegurar unigrama exacto ---
EXACT_UNIGRAM_BONUS = 3.5 # empuja exactos (e.g., "agua" == "agua")
UNIGRAM_PREFER_BONUS = 0.8 # preferir nombres de 1 palabra
HEAD_POS_BONUS = 0.25 # si el token está al inicio/fin del compuesto
LONG_COMPOUND_PENALTY = 0.40 # penaliza >=3 tokens
ADJ_VERB_NOUN_PAIR_BONUS = 0.15 # bonus para pares verbo+sustantivo frecuentes

```

Explicación línea por línea:

Línea 1: # @title Configuración de rutas y parámetros del sistema

- Comentario que indica que este bloque se encarga de la configuración de rutas y parámetros.

Línea 2: from pathlib import Path

- Importa la clase Path de la librería pathlib, que se utiliza para manipular rutas de archivos y directorios.

Líneas 4-8: Definición de rutas de entrada

- Se definen las rutas de los archivos XML para los dominios de salud, educación y cultura, y la ruta de la carpeta que contiene todos los pictogramas.
- Se utiliza rstrip("/") para eliminar cualquier barra final en las rutas, asegurando un formato consistente.

Líneas 10-13: Definición de rutas de salida

- Se definen las rutas de los archivos JSON de salida para cada dominio, que contendrán los candidatos de pictogramas.

Línea 16: CACHE_EMB_PICTOS = "/content/emb_pictos_v3.npy"

- Define la ruta del archivo de cache para almacenar los embeddings de las etiquetas de los pictogramas, lo que acelera las ejecuciones posteriores.

Línea 19: MODEL_NAME = "dccuchile/bert-base-spanish-wwm-cased"

- Especifica el nombre del modelo pre-entrenado de BERT en español que se utilizará para generar los embeddings.

Líneas 22-24: Parámetros de ranking

- K_POR_TOKEN = 6: Número máximo de candidatos a retener por token (top-K).
- M_POOL = 60: Tamaño del pool léxico inicial de candidatos.
- UMBRAL_SCORE = 0.52: Umbral mínimo de similitud para aceptar un candidato (si el top-1 no alcanza este umbral, se omite).

Líneas 27-29: Reglas para tokens cortos y fuzzy matching

- LONG_MIN_FUZZY = 6: Longitud mínima del token para aplicar fuzzy matching (evita coincidencias en palabras cortas).
- TOKEN_CORTO_EXACTO = 3: Si la longitud del token es menor o igual a 3, solo se aceptan coincidencias exactas o sinónimos.

Líneas 32-36: Configuración de stopwords

- OMITIR_STOPWORDS_EN_SALIDA = True: Si es True, las stopwords no se incluyen en la salida JSON.
- STOP_TOKENS: Conjunto de stopwords y signos de puntuación que se omitirán en la salida.
- Sistema de Allowlist: Stopwords importantes como "sí", "no", "hoy", "ayer" ahora se incluyen en la salida, aunque sean stopwords

Líneas 39-48: Sinónimos base

- SINONIMOS_BASE: Diccionario que mapea palabras a conjuntos de sinónimos. Se utiliza para expandir las búsquedas.

Líneas 51-54: Semillas de n-gramas

- SEMILLAS_NGRAM: Conjunto de frases (n-gramas) que se priorizan si existen en la base de datos de pictogramas.

Líneas 58-61: Filtros de contenidos sensibles

- BLOCKLIST_GLOBAL: Conjunto de términos globales que se bloquearán en todos los dominios.
- BLOCKLIST_POR_DOMINIO: Diccionario que especifica listas de bloqueo por dominio, combinando la lista global con términos específicos.

Líneas 64-68: Boost por dominio

- BOOST_DOMINIO: Diccionario que por cada dominio define un conjunto de palabras que, si aparecen en la etiqueta del pictograma, aumentan su puntuación.

Líneas 71-74: Coherencia contextual (bonos)

- W_COOCC = 0.06: Peso del bono por co-ocurrencia con otras palabras de la oración.
- W_BIGRAM = 0.08: Peso del bono si la etiqueta contiene un bigrama relevante de la oración.
- W_BOOST_DOM = 0.03: Peso del bono por palabras de dominio.
- Bonos para Unigramas Exactos: Sistema de scoring favorece coincidencias exactas “agua” == “agua”

Línea	Explicación
from pathlib import Path	Importa la librería Path para manejo de rutas de archivos
RUTA_XML_SALUD = "...".rstrip("/")	Define y sanitiza la ruta del archivo XML de salud eliminando "/" final
RUTA_XML_EDUCACION = "...".rstrip("/")	Define y sanitiza la ruta del archivo XML de educación
RUTA_XML_CULTURA = "...".rstrip("/")	Define y sanitiza la ruta del archivo XML de cultura
RUTA_PICTOS = "...".rstrip("/")	Define y sanitiza la ruta del directorio de pictogramas
OUT_SALUD = "...".json	Define la ruta de salida para candidatos de salud
OUT_EDUCACION = "...".json	Define la ruta de salida para candidatos de educación
OUT_CULTURA = "...".json	Define la ruta de salida para candidatos de cultura
CACHE_EMB_PICTOS = "...".npy	Define la ruta del archivo cache para embeddings de pictogramas
MODEL_NAME = "dccuchile/..."	Especifica el modelo BERT en español a utilizar
K_POR_TOKEN = 6	Define el número máximo de candidatos a retener por token
M_POOL = 60	Define el tamaño del pool inicial de candidatos léxicos
UMBRAL_SCORE = 0.52	Establece el umbral mínimo de similitud para aceptar candidatos
LONG_MIN_FUZZY = 6	Define longitud mínima para aplicar búsqueda difusa
TOKEN_CORTO_EXACTO = 3	Establece que tokens ≤ 3 caracteres solo busquen coincidencias exactas
OMITIR_STOPWORDS_EN_SALIDA = True	Configura si excluir stopwords del resultado final
STOP_TOKENS = set(""...".split())	Define conjunto de stopwords y signos de puntuación a omitir
ALLOWLIST_STOPWORDS_RAW = {...}	Nueva lista de stopwords importantes que SÍ deben exportarse
ALLOWLIST_STOPWORDS = None	Variable para almacenar versión normalizada (se inicializa después)
FORZAR_INCLUSION_ALLOWLIST_VACIOS = True	Fuerza inclusión de stopwords allowlist incluso sin pictogramas

SINONIMOS_BASE = {...}	Diccionario de sinónimos para expansión de búsquedas
SEMILLAS_NGRAM = {...}	Conjunto de frases comunes que se priorizan en búsquedas
BLOCKLIST_GLOBAL = {...}	Lista global de términos sensibles a bloquear
BLOCKLIST_POR_DOMINIO = {...}	Diccionario de listas de bloqueo específicas por dominio
BOOST_DOMINIO = {...}	Diccionario de palabras que aumentan score por dominio
W_COOCC = 0.06	Peso del bono por co-ocurrencia contextual
W_BIGRAM = 0.08	Peso del bono por coincidencia de bigramas
W_BOOST_DOM = 0.03	Peso del bono por palabras de dominio relevantes
EXACT_UNIGRAM_BONUS = 3.5	Nuevo bono grande para coincidencias exactas de unigramas
UNIGRAM_PREFER_BONUS = 0.8	Bono para preferir etiquetas de una sola palabra
HEAD_POS_BONUS = 0.25	Bono si el token está en posición inicial/final de compuestos
LONG_COMPOUND_PENALTY = 0.40	Penalización para etiquetas con 3+ tokens
ADJ_VERB_NOUN_PAIR_BONUS = 0.15	Bono para pares verbo+sustantivos frecuentes

Entradas:

- Rutas predefinidas a archivos XML de entrada
- Configuraciones de parámetros de ranking y filtrado
- Diccionarios de sinónimos, listas de bloqueo y palabras de boost
- Configuraciones de pesos para algoritmos de scoring

Salidas:

- Variables globales configuradas para ser utilizadas en los módulos posteriores
- Estructuras de datos inicializadas para procesamiento lingüístico
- Configuración de bonos y penalizaciones para matching exacto
- Este bloque de código no ejecuta ninguna operación más allá de la definición de variables, por lo que no produce salidas visibles inmediatas, pero establece el estado inicial para el resto del proceso.

Bloque 3: Utilidades de Normalización, Lectura, Lemas y POS (spaCy opcional)

Propósito del Bloque: Proporcionar funciones utilitarias para procesamiento de texto, incluyendo normalización, lectura de archivos XML, tokenización y lematización usando spaCy de manera opcional.

```

import os, re, json, numpy as np
import xml.etree.ElementTree as ET
from collections import defaultdict
from typing import List, Dict, Tuple, Set
from unicode import unicode
from rapidfuzz import fuzz

# spaCy opcional (si no está, seguimos sin lematización/POS)
USE_SPACY = True
nlp = None
if USE_SPACY:
    try:
        import spacy
        nlp = spacy.load("es_core_news_sm")
    except Exception as e:
        print("spaCy no disponible. Continuando sin lemas/POS. Detalle:", e)
        nlp = None

def norm_text(s: str) -> str:
    """ Normaliza texto: minúsculas, sin tildes, espacios simples. """
    s = s.strip().lower().replace("_", " ")
    s = re.sub(r"\s+", " ", s)
    s = unicode(s)
    return s

def es_stop_or_punt(token: str) -> bool:
    t = norm_text(token)
    return (t in STOP_TOKENS) or (len(t) == 0)

def tokens_de_etiqueta(et_norm: str) -> List[str]:
    """ Tokeniza etiquetas normalizadas en [a-z0-9]+ (palabras). """
    return re.findall(r"[a-z0-9]+", et_norm)

def leer_oraciones_xml(path_xml: str) -> List[str]:
    """ Lee <frase> del XML y devuelve lista de oraciones. """
    tree = ET.parse(path_xml)
    root = tree.getroot()
    frases = []
    for fr in root.iter("frase"):
        texto = (fr.text or "").strip()
        texto = re.sub(r"\s+", " ", texto)
        if texto:
            frases.append(texto)
    return frases

def lematiza_tokens(tokens: List[str]) -> List[str]:
    """ Devuelve lemas si spaCy está disponible; si no, devuelve los mismos tokens. """
    if nlp is None:
        return tokens
    doc = nlp(" ".join(tokens))
    lem = []
    j = 0
    for tok in doc:
        # preserva segmentación original por espacios usando índice simple
        if tok.text.strip()!="":
            continue
        lem.append(tok.lemma_)
        j += 1
    return lem

```

Explicación Línea por Línea:

Línea	Explicación
import os, re, json, numpy as np	Importa módulos del sistema, expresiones regulares, JSON y numpy
import xml.etree.ElementTree as ET	Importa parser XML para leer archivos de oraciones
from collections import defaultdict	Importa diccionario con valores por defecto
from typing import List, Dict, Tuple, Set	Importa anotaciones de tipo para documentación

from unidecode import unidecode	Importa función para eliminar acentos y caracteres especiales
from rapidfuzz import fuzz	Importa funciones para comparación difusa de cadenas
USE_SPACY = True	Bandera para habilitar/deshabilitar uso de spaCy
nlp = None	Inicializa variable del procesador de spaCy como nula
if USE_SPACY:	Verifica si spaCy está habilitado
try:	Intenta cargar spaCy con manejo de errores
import spacy	Importa la librería spaCy
nlp = spacy.load("es_core_news_sm")	Carga el modelo de lenguaje español de spaCy
except Exception as e:	Captura cualquier error en la carga de spaCy
print("spaCy no disponible...")	Mensaje informativo si spaCy no está disponible
nlp = None	Asegura que nlp sea None en caso de error
def norm_text(s: str) -> str:	Define función de normalización de texto
s = s.strip().lower().replace("_", " ")	Elimina espacios, convierte a minúsculas y reemplaza guiones bajos
s = re.sub(r"\s+", " ", s)	Normaliza espacios múltiples a espacio simple
s = unidecode(s)	Elimina acentos y caracteres especiales
return s	Retorna texto normalizado
global ALLOWLIST_STOPWORDS	Declara variable global para la allowlist normalizada
ALLOWLIST_STOPWORDS = { norm_text(w) for w in ALLOWLIST_STOPWORDS_RAW }	Crea conjunto normalizado de stopwords importantes
def es_puntuacion(token: str) -> bool:	Nueva función para detectar tokens que son solo puntuación
return bool(re.fullmatch(r"[\W_]+", token))	Usa regex para identificar puntuación pura
def es_stop(token: str) -> bool:	Función modificada con lógica de allowlist
t = norm_text(token)	Normaliza el token de entrada
return (t in STOP_TOKENS) and (t not in ALLOWLIST_STOPWORDS)	Lógica actualizada: solo es stop si está en STOP_TOKENS Y NO en allowlist
def es_stop_or_punt(token: str) - > bool:	Función que combina ambas verificaciones, para detectar stopwords o puntuación

return es_puntuacion(token) or es_stop(token)	Retorna True si es puntuación O stopword (excluyendo allowlist)
def tokens_de_etiqueta(et_norm: str)...	Función para tokenizar etiquetas normalizadas
return re.findall(r"[a-z0-9]+", et_norm)	Extrae palabras usando regex (letras y números)
def leer_oraciones_xml(path_xml: str)...	Función para leer oraciones desde archivo XML
tree = ET.parse(path_xml)	Parsea el archivo XML
root = tree.getroot()	Obtiene el elemento raíz del XML
frases = []	Inicializa lista vacía para almacenar frases
for fr in root.iter("frase"):	Itera sobre todos los elementos "frase" en el XML
texto = (fr.text or "").strip()	Extrae y limpia el texto de cada frase
texto = re.sub(r"\s+", " ", texto)	Normaliza espacios en el texto
if texto:	Verifica que el texto no esté vacío
frases.append(texto)	Agrega la frase a la lista
return frases	Retorna la lista completa de frases
def lematiza_tokens(tokens: List[str])...	Función para lematizar tokens
if nlp is None:	Verifica si spaCy está disponible
return tokens	Retorna tokens originales si spaCy no está disponible
doc = nlp(" ".join(tokens))	Procesa los tokens unidos como texto con spaCy
lem = []	Inicializa lista para lemas
for tok in doc:	Itera sobre tokens procesados por spaCy
if tok.text.strip() == "":	Omite tokens vacíos
continue	Continúa con siguiente token
lem.append(tok.lemma_)	Agrega el lema del token a la lista
return lem	Retorna lista de lemas

Entradas:

- Texto sin procesar (para normalización)

- Tokens individuales (para detección de stopwords)
- Rutas de archivos XML (para lectura de oraciones)
- Listas de tokens (para lematización)

Salidas:

- Texto normalizado
- Valores booleanos (detección de stopwords)
- Listas de tokens
- Listas de oraciones extraídas de XML
- Listas de lemas (si spaCy está disponible)

Bloque 4: Cargar BETO y Definir Funciones de Embeddings

Propósito del Bloque: Cargar el modelo BETO (BERT en español) y definir funciones especializadas para generar embeddings tanto a nivel de oración completa (CLS) como a nivel de tokens individuales en contexto.

```
import torch
from transformers import AutoTokenizer, AutoModel
from tqdm.auto import tqdm

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print("Dispositivo:", DEVICE)

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME, use_fast=True)
model = AutoModel.from_pretrained(MODEL_NAME, output_hidden_states=True).to(DEVICE).eval()

@torch.no_grad()
def embed_sentence_cls(textos: List[str] | str) -> np.ndarray:
    """
    Embedding CLS (promedio últimas 4 capas), normalizado L2.
    Acepta lista o string. Devuelve (N,768) o (768,) si N=1.
    """
    single = False
    if isinstance(textos, str):
        textos = [textos]
        single = True

    vs = []
    bs = 64
    for i in tqdm(range(0, len(textos), bs), desc="Embeddings CLS (batch)"):
        batch = textos[i:i+bs]
        enc = tokenizer(batch, return_tensors='pt', padding=True, truncation=True, max_length=128).to(DEVICE)
        out = model(**enc)
        hs = out.hidden_states
        cls = torch.stack([hs[-1][:,0], hs[-2][:,0], hs[-3][:,0], hs[-4][:,0]], dim=0).mean(dim=0) # (8,768)
        v = torch.nn.functional.normalize(cls, p=2, dim=1)
        vs.append(v.detach().cpu().numpy())
    V = np.vstack(vs)
    return V[0] if (single and V.shape[0] == 1) else V
```

```

@torch.no_grad()
def embed_tokens_en_context(oracion: str, palabras: List[str]) -> List[np.ndarray]:
    """
    Para cada palabra visible en la oración, obtiene el embedding contextual
    promediando sus subpalabras (últimas 4 capas). Si no hay alineación,
    usa CLS de la oración como fallback.
    """
    enc = tokenizer(
        oracion,
        return_tensors="pt",
        return_offsets_mapping=True,
        add_special_tokens=True,
        truncation=True,
        max_length=256,
    )
    offs = enc.pop("offset_mapping")[0].tolist()
    enc_model = {k: v.to(DEVICE) for k, v in enc.items()}
    out = model(**enc_model)
    hs = out.hidden_states

    # spans de palabras por búsqueda secuencial
    spans_pal, idx = [], 0
    for w in palabras:
        j = oracion.lower().find(w.lower(), idx)
        if j == -1: j = idx
        spans_pal.append((j, j+len(w)))
        idx = j + len(w)

    token_vecs = []
    for (a, b) in spans_pal:
        sub_idx = [i for i, (x, y) in enumerate(offs) if not (x == 0 and y == 0) and (max(a, x) < min(b, y))]
        if not sub_idx:
            v = embed_sentence_cls(oracion) # (768,)
            token_vecs.append(v)
            continue
        sub_stack = []
        for layer in (-1, -2, -3, -4):
            layer_out = hs[layer].squeeze(0) # (seq,768)
            sel = layer_out[sub_idx, :].mean(dim=0)
            sub_stack.append(sel)
        v = torch.stack(sub_stack, dim=0).mean(dim=0)
        v = torch.nn.functional.normalize(v, p=2, dim=0)
        token_vecs.append(v.detach().cpu().numpy())
    return token_vecs

```

Explicación Línea por Línea:

Línea	Explicación
import torch	Importa PyTorch para computación tensorial
from transformers import AutoTokenizer, AutoModel	Importa las clases para cargar el tokenizador y el modelo.
from tqdm.auto import tqdm	Importa la barra de progreso.

DEVICE = "cuda" ...	Detecta automáticamente GPU (CUDA) o usa CPU
print("Dispositivo:", DEVICE)	Muestra el dispositivo que se está usando.
tokenizer = AutoTokenizer.from_pretrained(...)	Carga el tokenizador rápido del modelo BETO
model = AutoModel.from_pretrained(...)	Carga el modelo BETO configurado para devolver capas ocultas
@torch.no_grad()	Decorador para deshabilitar el cálculo de gradientes (ahorro de memoria).
def embed_sentence_cls(...) -> ...:	Define función para embeddings de oraciones completas
single = False	Bandera para indicar si la entrada es una sola cadena.
if isinstance(textos, str):	Verifica si la entrada es una cadena (string simple).
textos = [textos]	Convierte la cadena en una lista de un elemento.
single = True	Establece la bandera a True que Marca que la entrada era única
vs = []	Lista para acumular vectores de embeddings por lote.
bs = 64	Tamaño del lote para procesamiento (batch size).
for i in tqdm(...):	Itera sobre los lotes con barra de progreso.
batch = textos[i:i+bs]	Obtiene un lote de textos.
enc = tokenizer(...)	Tokeniza el lote, con padding y truncamiento.
out = model(**enc)	Pasa el lote tokenizado al modelo.
hs = out.hidden_states	Obtiene las capas ocultas del modelo.
cls = torch.stack(...).mean(dim=0)	Combina las 4 últimas capas y promedia para obtener el embedding CLS.
v = torch.nn.functional.normalize(...)	Normaliza vectores a norma L2 (cosine similarity)

vs.append(v.detach().cpu().numpy())	Añade el embedding a la lista (convertido a numpy).
V = np.vstack(vs)	Concatena todos los embeddings de los lotes.
return V[o] if ... else V	Retorna el primer elemento si es una sola cadena, sino la matriz.
def embed_tokens_en_context(...) -> ...:	Define la función para embeddings contextuales de tokens.
enc = tokenizer(..., return_offsets_mapping=True, ...)	Tokeniza la oración preservando mapeo de posiciones de caracteres.
offs = enc.pop("offset_mapping")[o].tolist()	Extrae los offsets y los convierte a lista.
enc_model = {k: v.to(DEVICE) ...}	Mueve los inputs del tokenizador al dispositivo.
out = model(**enc_model)	Ejecuta modelo para obtener embeddings contextuales
hs = out.hidden_states	Extrae capas ocultas
spans_pal, idx = [], o	Inicializa la lista de spans y el índice de búsqueda.
for w in palabras:	Itera sobre cada palabra objetivo.
j = oracion.lower().find(w.lower(), idx)	Busca la palabra en la oración (ignorando mayúsculas).
if j == -1: j = idx	Si no la encuentra, usa el índice actual.
spans_pal.append((j, j+len(w)))	Guarda el span (inicio, fin) de la palabra.
idx = j + len(w)	Actualiza el índice para la siguiente búsqueda.
token_vecs = []	Lista para los embeddings de cada palabra (tokens).
for (a, b) in spans_pal:	Itera sobre los spans de las palabras.
sub_idxs = [i for i, (x, y) ...]	Encuentra los índices de tokens que se solapan con el span.
if not sub_idxs:	Si no hay tokens que se solapen, usa el embedding CLS.
v = embed_sentence_cls(oracion)	Calcula el embedding CLS de la oración.

token_vecs.append(v)	Añade el embedding CLS.
continue	Continúa con la siguiente palabra.
sub_stack = []	Lista para los promedios de cada capa.
for layer in (-1, -2, -3, -4):	Itera sobre las últimas 4 capas.
layer_out = hs[layer].squeeze(0)	Elimina la dimensión del batch.
sel = layer_out[sub_idx, :].mean(dim=0)	Promedia los embeddings de los tokens de la palabra en la capa.
sub_stack.append(sel)	Añade el promedio de la capa.
v = torch.stack(sub_stack, dim=0).mean(dim=0)	Promedia los promedios de las capas.
v = torch.nn.functional.normalize(v, p=2, dim=0)	Normaliza el embedding.
token_vecs.append(v.detach().cpu().numpy())	Añade el embedding a la lista.
return token_vecs	Retorna lista de embeddings contextuales

Entradas:

- MODEL_NAME predefinido (del bloque anterior)
- Textos u oraciones para generar embeddings
- Listas de palabras específicas para embeddings contextuales

Salidas:

- Modelo BETO y tokenizador cargados en memoria
- Funciones embed_sentence_cls() y embed_tokens_en_context() disponibles
- Vectores de embeddings normalizados (768 dimensiones)

Bloque 5: Indexar Pictogramas y Construir Embeddings de Etiquetas

Propósito del Bloque: Cargar todos los pictogramas disponibles, indexar sus etiquetas y generar embeddings semánticos para cada una, permitiendo búsquedas eficientes por palabras individuales, frases completas y similitud semántica.

```
# @title Indexar pictogramas y construir embeddings de etiquetas
def cargar_pictos_y_indexar(dir_pictos: str):
    """
    Retorna:
    - etiquetas_txt[id] -> etiqueta original (con _)
    - etiquetas_norm[id] -> etiqueta normalizada (espacios)
    - idx_token[token] -> set(ids) por palabra completa
    - idx_frase[frase_norm] -> set(ids) por frase completa (multi-palabra)
    """
    etiquetas_txt: Dict[str, str] = {}
    etiquetas_norm: Dict[str, str] = {}
    idx_token: Dict[str, Set[str]] = defaultdict(set)
    idx_frase: Dict[str, Set[str]] = defaultdict(set)

    files = [f for f in os.listdir(dir_pictos) if f.endswith(".jpg")]
    for fname in tqdm(files, desc="Indexando pictogramas"):
        try:
            pid, name = fname.split("-", 1)
            int(pid)
        except:
            continue
        etiqueta = name[:-4] # sin .jpg
        et_norm = norm_text(etiqueta)

        etiquetas_txt[pid] = etiqueta
        etiquetas_norm[pid] = et_norm

        toks = tokens_de_etiqueta(et_norm)
        for t in toks:
            idx_token[t].add(pid)

        if " " in et_norm:
            idx_frase[et_norm].add(pid)

    return etiquetas_txt, etiquetas_norm, idx_token, idx_frase
```

Línea	Explicación
def cargar_pictos_y_indexar(dir_pictos: str):	Define la función para indexar pictogramas
etiquetas_txt: Dict[str, str] = {}	Inicializa diccionario para etiquetas originales
etiquetas_norm: Dict[str, str] = {}	Inicializa diccionario para etiquetas normalizadas

idx_token: Dict[str, Set[str]] = defaultdict(set)	Inicializa índice de tokens con conjuntos
idx_frase: Dict[str, Set[str]] = defaultdict(set)	Inicializa índice de frases con conjuntos
files = [f for f in os.listdir(dir_pictos) if f.endswith(".jpg")]	Lista archivos .jpg en el directorio
for fname in tqdm(files, desc="Indexando pictogramas"):	Itera archivos con barra de progreso
try:	Intenta extraer ID y nombre del archivo
pid, name = fname.split("-", 1)	Separa el ID del resto del nombre (por el primer guion)
int(pid)	Verifica que el ID sea numérico (si no, salta)
except:	Si hay error, omite el archivo
continue	Continúa con el siguiente archivo
etiqueta = name[:-4]	Remueve la extensión .jpg para obtener la etiqueta
et_norm = norm_text(etiqueta)	Normaliza la etiqueta (minúsculas, sin tildes, etc.)
etiquetas_txt[pid] = etiqueta	Guarda etiqueta original por ID
etiquetas_norm[pid] = et_norm	Guarda etiqueta normalizada por ID
toks = tokens_de_etiqueta(et_norm)	Tokeniza la etiqueta normalizada en palabras
for t in toks:	Itera cada token (palabra) de la etiqueta
idx_token[t].add(pid)	Añade el ID al conjunto de ese token en el índice
if " " in et_norm:	Si la etiqueta normalizada tiene espacios (es una frase)
idx_frase[et_norm].add(pid)	Añade el ID al conjunto de esa frase en el índice
return etiquetas_txt, etiquetas_norm, idx_token, idx_frase	Retorna los diccionarios e índices

```
def construir_embeddings_etiquetas(etiquetas_txt: Dict[str, str], cache_path: str | None = None) -> Dict[str, np.ndarray]:
    """
    Construye embeddings CLS por etiqueta en batch y cachea en .npy.
    """
    emb: Dict[str, np.ndarray] = {}
    if cache_path and os.path.exists(cache_path):
        try:
            loaded = np.load(cache_path, allow_pickle=True).item()
            emb = {k: np.array(v) for k, v in loaded.items()}
            print(f"Cargado cache embeddings: {cache_path} ({len(emb)})")
        except Exception as e:
            print("Cache inválido, se reconstruirá:", e)

    faltan = [pid for pid in etiquetas_txt if pid not in emb]
    if faltan:
        textos = [etiquetas_txt[pid].replace("_", " ") for pid in faltan]
        V = embed_sentence_cls(textos) # (N,768)
        for pid, v in zip(faltan, V):
            emb[pid] = v
    if cache_path:
        np.save(cache_path, emb, allow_pickle=True)
        print(f"Embeddings guardados en cache: {cache_path} ({len(emb)})")
    return emb
```

Línea	Explicación
def construir_embeddings_etiquetas(...) -> Dict[str, np.ndarray]:	Define función para construir embeddings
emb: Dict[str, np.ndarray] = {}	Inicializa diccionario de embeddings
if cache_path and os.path.exists(cache_path):	Si hay cache y existe, intenta cargarlo
try:	Intenta cargar el cache
loaded = np.load(cache_path, allow_pickle=True).item()	Carga el diccionario desde el archivo .npy
emb = {k: np.array(v) for k, v in loaded.items()}	Convierte los valores a arrays numpy
print(f'Cargado cache embeddings: ...')	Mensaje de cache cargado
except Exception as e:	Si falla, imprime error y reconstruirá
print("Cache inválido, se reconstruirá:", e)	Informa que el cache no se pudo cargar
faltan = [pid for pid in etiquetas_txt if pid not in emb]	Lista de IDs que no están en el cache
if faltan:	Si hay IDs faltantes

textos = [etiquetas_txt[pid].replace("_", " ") for pid in faltan]	Prepara textos reemplazando _ por espacios
V = embed_sentence_cls(textos)	Genera embeddings para los textos faltantes
for pid, v in zip(faltan, V):	Itera sobre los IDs y sus embeddings
emb[pid] = v	Asigna el embedding al ID en el diccionario
if cache_path:	Si se proporcionó una ruta de cache
np.save(cache_path, emb, allow_pickle=True)	Guarda el diccionario de embeddings en cache
print(f'Embeddings guardados en cache: ...')	Mensaje de guardado
return emb	Retorna el diccionario de embeddings

```
# Ejecutar indexado + embeddings
etiquetas_txt, etiquetas_norm, idx_token, idx_frase = cargar_pictos_y_indexar(RUTA_PICTOS)
emb_etq = construir_embeddings_etiquetas(etiquetas_txt, CACHE_EMB_PICTOS)
print(f"Total pictos indexados: {len(etiquetas_txt)}")
```

Línea	Explicación
etiquetas_txt, ... = cargar_pictos_y_indexar(RUTA_PICTOS)	Ejecuta la indexación de pictogramas
emb_etq = construir_embeddings_etiquetas(...)	Construye los embeddings de las etiquetas
print(f"Total pictos indexados: ...")	Muestra el total de pictogramas indexados

Este bloque es fundamental para tener una base de datos de pictogramas indexada por palabras y frases, y sus correspondientes embeddings semánticos, que se utilizarán en las siguientes etapas para buscar los pictogramas más relevantes para cada oración.

Entradas:

- `dir_pictos`: Ruta al directorio que contiene los archivos de imagen de los pictogramas (con extensión .jpg)
- `cache_path`: Ruta opcional para un archivo de cache donde guardar/leer los embeddings precalculados

Salidas:

- etiquetas_txt: Diccionario que mapea ID de pictograma a su etiqueta original (con guiones bajos)
- etiquetas_norm: Diccionario que mapea ID de pictograma a su etiqueta normalizada (con espacios)
- idx_token: Índice inverso que mapea cada token (palabra) a un conjunto de IDs de pictogramas que contienen esa palabra
- idx_frase: Índice inverso que mapea cada frase normalizada (etiqueta completa) a un conjunto de IDs de pictogramas
- emb_etq: Diccionario que mapea ID de pictograma a su embedding (vector de 768 dimensiones)

Bloque 6: Detección de N-gramas, Pool Léxico y Filtros de Sensibilidad

Propósito del Bloque: Implementar funciones para detectar frases multi-palabra (n-gramas), construir pools de candidatos léxicos para tokens individuales y aplicar filtros de contenido sensible para garantizar resultados apropiados.

```
def detectar_ngramas(oracion_norm_toks: List[str],
                    idx_frase: Dict[str, Set[str]]) -> List[Tuple[int, int, str, List[str]]]:
    """
    Retorna spans [(i_ini, i_fin_excl, frase_norm, ids)] ordenados y NO solapados.
    Ventanas 4,3,2. Acepta exacto si está en idx_frase o si es semilla muy parecida.
    """
    n = len(oracion_norm_toks)
    spans = []
    ocupado = [False]*n
    frases_bd = set(idx_frase.keys())

    for L in [4,3,2]:
        i = 0
        while i <= n - L:
            if any(ocupado[i:i+L]):
                i += 1; continue
            frag_norm = " ".join(oracion_norm_toks[i:i+L])
            ids = None

            if frag_norm in frases_bd:
                ids = list(idx_frase[frag_norm])
            elif frag_norm in SEMILLAS_NGRAM:
                # buscar la frase más similar en BD
                mejor, best = None, 0
                for f in frases_bd:
                    r = fuzz.ratio(frag_norm, f)
                    if r > best:
                        best, mejor = r, f
                if best >= 90 and mejor is not None:
                    ids = list(idx_frase[mejor])
                    frag_norm = mejor

            if ids:
                spans.append((i, i+L, frag_norm, ids))
                for k in range(i, i+L):
                    ocupado[k] = True
                i += L
            else:
                i += 1

    spans.sort(key=lambda x: x[0])
    return spans
```

Explicación Línea por Línea:

Función detectar_ngramas

Línea	Explicación
def detectar_ngramas(...):	Define la función para detectar n-gramas (frases multi-palabra) en la oración.
n = len(oracion_norm_toks)	Longitud de la lista de tokens.
spans = []	Inicializa lista para almacenar los n-gramas detectados.
ocupado = [False]*n	Array para marcar tokens ya cubiertos por un n-grama.
frases_bd = set(idx_frase.keys())	Conjunto de frases completas en la base de datos de pictogramas.
for L in [4,3,2]:	Itera sobre longitudes de n-grama (de 4 a 2 palabras).
i = 0	Inicializa índice de inicio de ventana.
while i <= n - L:	Itera hasta donde cabe una ventana de longitud L.
if any(ocupado[i:i+L]):	Si algún token en la ventana ya está cubierto, salta.
i += 1; continue	Avanza al siguiente token.
frag_norm = " ".join(oracion_norm_toks[i:i+L])	Construye la frase normalizada del fragmento.
ids = None	Inicializa variable para IDs de pictogramas.
if frag_norm in frases_bd:	Si la frase está en la base de datos, obtiene sus IDs.
ids = list(idx_frase[frag_norm])	Asigna los IDs de la frase.
elif frag_norm in SEMILLAS_NGRAM:	Si no está en BD pero es una semilla, busca la más similar en BD.
mejor, best = None, 0	Inicializa mejor coincidencia y score.
for f in frases_bd:	Itera sobre todas las frases en la base de datos.
r = fuzz.ratio(frag_norm, f)	Calcula similitud entre la frase y la de BD.
if r > best:	Si es la mejor similitud, actualiza.
best, mejor = r, f	Guarda el mejor score y la frase.

if best >= 90 and mejor is not None:	Si la similitud es alta (>=90%), usa esa frase.
ids = list(idx_frase[mejor])	Toma los IDs de la frase similar.
frag_norm = mejor	Actualiza la frase normalizada a la similar.
if ids:	Si se encontraron IDs, agrega el span.
spans.append((i, i+L, frag_norm, ids))	Agrega el n-grama a la lista.
for k in range(i, i+L):	Marca los tokens como cubiertos.
ocupado[k] = True	Actualiza el array ocupado.
i += L	Avanza el índice por la longitud del n-grama.
else:	Si no se encontró, avanza de a un token.
i += 1	Incrementa el índice.
spans.sort(key=lambda x: x[0])	Ordena los spans por posición de inicio.
return spans	Retorna la lista de n-gramas detectados.

```
def _token_pasa_blocklist(et_norm: str, dominio: str|None) -> bool:
    """ True si la etiqueta NO contiene términos bloqueados. """
    bl = set(BLOCKLIST_GLOBAL)
    if dominio and dominio in BLOCKLIST_POR_DOMINIO:
        bl |= BLOCKLIST_POR_DOMINIO[dominio]
    en = et_norm
    return not any(bad in en for bad in bl)
```

Función _token_pasa_blocklist

Línea	Explicación
def _token_pasa_blocklist(...):	Función interna que verifica si una etiqueta pasa el filtro de contenido sensible.
bl = set(BLOCKLIST_GLOBAL)	Inicializa el conjunto de términos bloqueados con la lista global.
if dominio and dominio in BLOCKLIST_POR_DOMINIO:	Si se especifica un dominio y tiene una lista específica, la agrega.
`bl	= BLOCKLIST_POR_DOMINIO[dominio]` Une las listas de bloqueo.
en = et_norm	Asigna la etiqueta normalizada a una variable local.
return not any(bad in en for bad in bl)	Retorna True si la etiqueta NO contiene ningún término bloqueado.

```

def pool_lexico(token: str,
                idx_token: Dict[str, Set[str]],
                etiquetas_norm: Dict[str, str],
                M: int=M_POOL,
                dominio: str|None=None) -> List[str]:
    """
    Pool robusto por palabra completa:
    - Exacto por token
    - Sinónimos (exactos)
    - Fuzzy contra tokens (umbral alto) SOLO si len(token)>=LONG_MIN_FUZZY
    - Filtro de sensibilidad (blocklist)
    """
    t = norm_text(token)
    cand: Set[str] = set()

    # exacto
    if t in idx_token:
        cand |= idx_token[t]

    # sinónimos exactos
    for s in SINONIMOS_BASE.get(t, set()):
        if s in idx_token:
            cand |= idx_token[s]

    # fuzzy estricto (token-level) sólo si el token es suficientemente largo
    if len(t) >= LONG_MIN_FUZZY and len(cand) < M:
        scored = []
        for pid, en in etiquetas_norm.items():
            if not _token_pasa_blocklist(en, dominio):
                continue
            toks = tokens_de_etiqueta(en)
            best = max((fuzz.ratio(t, tok) for tok in toks), default=0)
            # umbral alto + control de longitud (evita piano->pino)
            if best >= 92 and abs(len(t) - max((len(tok) for tok in toks), default=len(t))) <= 1:
                scored.append((best, pid))
        scored.sort(reverse=True)
        for _, pid in scored:
            cand.add(pid)
            if len(cand) >= M:
                break

    # aplicar blocklist también al conjunto exacto
    cand_filtrado = [pid for pid in cand if _token_pasa_blocklist(etiquetas_norm[pid], dominio)]
    return cand_filtrado[:max(M, len(cand_filtrado))]

```

Función pool_lexico

Línea	Explicación
def pool_lexico(...):	Define la función para construir un pool de candidatos para un token.
t = norm_text(token)	Normaliza el token.
cand: Set[str] = set()	Inicializa un conjunto para los candidatos.
if t in idx_token:	Si el token está en el índice invertido, agrega sus IDs.
`cand	= idx_token[t]` Une los conjuntos.
for s in SINONIMOS_BASE.get(t, set()):	Itera sobre los sinónimos del token.
if s in idx_token:	Si el sinónimo está en el índice, agrega sus IDs.
`cand	= idx_token[s]` Une los conjuntos.
if len(t) >= LONG_MIN_FUZZY and len(cand) < M:	Si el token es largo y hay pocos candidatos, usa búsqueda difusa.

scored = []	Lista para candidatos con score.
for pid, en in etiquetas_norm.items():	Itera sobre todas las etiquetas normalizadas.
if not _token_pasa_blocklist(en, dominio):	Si no pasa el filtro, omite.
continue	Salta a la siguiente etiqueta.
toks = tokens_de_etiqueta(en)	Tokeniza la etiqueta.
best = max((fuzz.ratio(t, tok) for tok in toks), default=0)	Calcula la máxima similitud entre el token y los tokens de la etiqueta.
if best >= 92 and ... <= 1:	Si la similitud es alta y la diferencia de longitud es <=1, lo considera.
scored.append((best, pid))	Agrega a la lista de scored.
scored.sort(reverse=True)	Ordena de mayor a menor score.
for _, pid in scored:	Itera sobre los candidatos ordenados.
cand.add(pid)	Agrega el candidato.
if len(cand) >= M:	Si se alcanza el máximo, termina.
break	Rompe el bucle.
cand_filtrado = ...	Filtra los candidatos que no pasan el blocklist.
return cand_filtrado[:max(M, len(cand_filtrado))]	Retorna hasta M candidatos.

Este bloque es crucial para la fase de recuperación de candidatos, combinando búsqueda exacta, por sinónimos y difusa, junto con filtros de seguridad.

Entradas:

- oracion_norm_toks: Lista de tokens normalizados de la oración.
- idx_frase: Índice invertido de frases normalizadas a conjuntos de IDs de pictogramas.
- token: Token individual a buscar.
- idx_token: Índice invertido de tokens a conjuntos de IDs de pictogramas.
- etiquetas_norm: Diccionario de IDs de pictogramas a etiquetas normalizadas.
- M: Tamaño máximo del pool léxico (por defecto M_POOL).
- dominio: Dominio actual (salud, educación, cultura) para filtrado.

Salidas:

- detectar_ngramas: Lista de tuplas (inicio, fin, frase_normalizada, IDs) de n-gramas detectados.
- pool_lexico: Lista de IDs de pictogramas candidatos para un token.

Bloque 7: Scoring Compuesto con Coherencia Contextual y Dominio

Propósito del Bloque: Definir las funciones de scoring que combinan múltiples señales (similitud semántica, similitud léxica, coherencia contextual y dominio) para rankear los pictogramas candidatos para cada token.

```
# @title Scoring compuesto con coherencia contextual y dominio
def score_lexico(token: str, etiqueta_norm: str) -> float:
    t = norm_text(token)
    toks = tokens_de_etiqueta(etiqueta_norm)
    if t in toks:
        return 1.0
    if t in SINONIMOS_BASE and any(s in toks for s in SINONIMOS_BASE[t]):
        return 0.9
    best = max((fuzz.ratio(t, tok) for tok in toks), default=0) / 100.0
    return min(best, 0.70)

def penalty_pos(token: str, etiqueta_norm: str) -> float:
    t = norm_text(token)
    e0 = tokens_de_etiqueta(etiqueta_norm)[:1]
    e0 = e0[0] if e0 else ""
    es_verbo_t = t.endswith(("ar", "er", "ir")) or t in {"ser", "estar", "haber", "ir"}
    es_verbo_e = e0.endswith(("ar", "er", "ir"))
    return 1.0 if es_verbo_t and not es_verbo_e else 0.0

def bonus_coocurrencia(etiqueta_norm: str, tokens_oracion_set: Set[str], token_actual: str) -> float:
    """
    Bono pequeño por co-ocurrencias: cuántas palabras de la oración (excluyendo el token actual y stopwords)
    aparecen también en la etiqueta.
    """
    etoks = set(tokens_de_etiqueta(etiqueta_norm))
    otros = tokens_oracion_set - {norm_text(token_actual)}
    overlap = len(etoks & otros)
    return min(2, overlap) * W_COOCC # máx 2*W_COOCC

def bonus_bigrama(etiqueta_norm: str, bigramas_oracion: Set[Tuple[str, str]]) -> float:
    """
    Bono si la etiqueta contiene algún bigrama relevante de la oración.
    """
    etoks = tokens_de_etiqueta(etiqueta_norm)
    et_bi = set(zip(etoks, etoks[1:])) if len(etoks) >= 2 else set()
    return W_BIGRAM if len(et_bi & bigramas_oracion) > 0 else 0.0

def bonus_dominio(etiqueta_norm: str, dominio: str|None) -> float:
    if not dominio or dominio not in BOOST_DOMINIO:
        return 0.0
    etoks = set(tokens_de_etiqueta(etiqueta_norm))
    bd = set(BOOST_DOMINIO[dominio])
    overlap = len(etoks & bd)
    return min(2, overlap) * W_BOOST_DOM

def combinar_score(sim_emb, sim_ctx, lex, pen, b_cooc, b_bi, b_dom):
    # pesos base + bonos
    return (0.50*sim_emb + 0.25*sim_ctx + 0.15*lex - 0.04*pen) + b_cooc + b_bi + b_dom
```

Explicación Línea por Línea:

Línea	Explicación
def score_lexico(token: str, etiqueta_norm: str) -> float:	Función para scoring léxico basado en coincidencia exacta, sinónimos o similitud difusa
t = norm_text(token)	Normaliza el token de entrada
toks = tokens_de_etiqueta(etiqueta_norm)	Extrae tokens de la etiqueta normalizada
if t in toks:	Si hay coincidencia exacta del token
return 1.0	Retorna puntuación máxima (1.0)
if t in SINONIMOS_BASE and any(...):	Si el token tiene sinónimos y alguno coincide
return 0.9	Retorna puntuación alta (0.9)
best = max((fuzz.ratio(t, tok)...)	Calcula máxima similitud difusa con tokens de etiqueta
return min(best, 0.70)	Retorna similitud (máximo 0.7 para coincidencias difusas)
def penalty_pos(token: str, etiqueta_norm: str) -> float:	Penalización por discrepancia de categoría gramatical
t = norm_text(token)	Normaliza token
eo = tokens_de_etiqueta(etiqueta_norm)[:1]	Toma primer token de la etiqueta
eo = eo[0] if eo else ""	Obtiene primer token o cadena vacía
es_verbo_t = t.endswith(...) or t in {...}	Detecta si token parece verbo (por terminación o verbos comunes)
es_verbo_e = eo.endswith(...)	Detecta si primer token de etiqueta parece verbo
return 1.0 if es_verbo_t and not es_verbo_e else 0.0	Penaliza si token es verbo pero etiqueta no
def bonus_coocurrencia(...):	Bono por palabras co-ocurrentes en oración y etiqueta
etoks = set(tokens_de_etiqueta(etiqueta_norm))	Convierte tokens de etiqueta a conjunto
otros = tokens_oracion_set - {norm_text(token_actual)}	Otros tokens de oración excluyendo el actual
overlap = len(etoks & otros)	Cuenta palabras en común

return min(2, overlap) * W_COOCC	Bono proporcional al overlap (máximo 2*W_COOCC)
def bonus_bigrama(...):	Bono por bigramas coincidentes
etoks = tokens_de_etiqueta(etiqueta_norm)	Tokeniza etiqueta
et_bi = set(zip(etoks, etoks[1:]))...	Genera bigramas de la etiqueta
return W_BIGRAM if len(et_bi & bigramas_oracion) > 0 else 0.0	Bono si hay bigramas en común
def bonus_dominio(...):	Bono por relevancia de dominio
if not dominio or dominio not in BOOST_DOMINIO:	Verifica dominio válido
return 0.0	Retorna 0 si no hay dominio válido
etoks = set(tokens_de_etiqueta(etiqueta_norm))	Convierte tokens de etiqueta a conjunto
bd = set(BOOST_DOMINIO[dominio])	Obtiene palabras clave del dominio
overlap = len(etoks & bd)	Cuenta palabras de dominio en etiqueta
return min(2, overlap) * W_BOOST_DOM	Bono proporcional al overlap (máximo 2*W_BOOST_DOM)
def combinar_score(...):	Combina todas las señales en puntuación final
return (0.50*sim_emb + 0.25*sim_ctx...	Fórmula ponderada: 50% embedding, 25% contexto, 15% léxico, -4% penalización, más bonos

Entradas:

- token: Token individual de la oración
- etiqueta_norm: Etiqueta normalizada del pictograma candidato
- tokens_oracion_set: Conjunto de tokens de la oración completa
- bigramas_oracion: Conjunto de bigramas extraídos de la oración
- dominio: Dominio específico (salud, educación, cultura)
- Puntuaciones individuales para combinación final

Salidas:

- score_lexico: Puntuación de similitud léxica (0.0-1.0)
- penalty_pos: Penalización por discrepancia de categoría gramatical (0.0-1.0)
- bonus_coocurrencia: Bono por palabras co-ocurrentes

- bonus_bigrama: Bono por bigramas coincidentes
- bonus_dominio: Bono por relevancia de dominio
- combinar_score: Puntuación final combinada

Bloque 8: Ranking Top-K y Pipeline por Oración

Propósito del Bloque: Implementar el pipeline completo de procesamiento por oración, integrando todas las funciones anteriores para generar los candidatos finales de pictogramas para cada token, aplicando ranking Top-K y considerando n-gramas, filtros de contenido y coherencia contextual.

```
# @title Ranking Top-K y pipeline por oración
def topk_candidatos_para_token(token: str,
                               vec_token: np.ndarray,
                               vec_oracion_cls: np.ndarray,
                               pool_ids: List[str],
                               emb_etq: Dict[str,np.ndarray],
                               etiquetas_norm: Dict[str,str],
                               etiquetas_txt: Dict[str,str],
                               tokens_oracion_set: Set[str],
                               bigramas_oracion: Set[tuple],
                               dominio: str|None,
                               K: int=K_POR_TOKEN) -> List[str]:

    if not pool_ids:
        return [""]

    scores = []
    for pid in pool_ids:
        en = etiquetas_norm[pid]
        if not _token_pasa_blocklist(en, dominio):
            continue
        ve = emb_etq[pid]
        sim_emb = float(np.dot(vec_token, ve))
        sim_ctx = float(np.dot(vec_oracion_cls, ve))
        lex = score_lexico(token, en)
        pen = penalty_pos(token, en)
        b_cooc = bonus_coocurrencia(en, tokens_oracion_set, token)
        b_bi = bonus_bigrama(en, bigramas_oracion)
        b_dom = bonus_dominio(en, dominio)
        s = combinar_score(sim_emb, sim_ctx, lex, pen, b_cooc, b_bi, b_dom)
        scores.append((s, pid))

    if not scores:
        return [""]

    scores.sort(reverse=True, key=lambda x: x[0])
    if scores[0][0] < UMBRAL_SCORE:
        return [""]

    out = []
    for _, pid in scores[:K]:
        out.append(f"{pid}-{etiquetas_txt[pid].replace(' ', '_')}.jpg")
    return out or [""]
```

```

def construir_bigrama_set(tokens_norm: List[str]) -> Set[Tuple[str, str]]:
    toks = [t for t in tokens_norm if t not in STOP_TOKENS and t]
    return set(zip(toks, toks[1:])) if len(toks) >= 2 else set()

def candidatos_para_oracion(oracion_str: str,
                            etiquetas_txt, etiquetas_norm, idx_token, idx_frase, emb_etq,
                            dominio: str=None,
                            k_por_token=K_POR_TOKEN, m_pool=M_POOL):
    """
    Devuelve (tokens_visibles, dict token->lista de candidatos) con n-gramas, filtros y ranking integrados.
    """
    # Tokens visibles (según el XML)
    toks_vis = [t for t in re.split(r"\s+", oracion_str.strip()) if t]

    # Lemmas opcionales para mejorar matching; si no, usa visibles normalizados
    toks_base = lematiza_tokens(toks_vis) if nlp is not None else toks_vis
    toks_norm = [norm_text(t) for t in toks_base]
    vec_cls = embed_sentence_cls(oracion_str)
    vecs_tok = embed_tokens_en_context(oracion_str, toks_vis)

    # Sets de coherencia contextual
    tokens_oracion_set = set([t for t in toks_norm if t and t not in STOP_TOKENS])
    bigramas_oracion = construir_bigrama_set(toks_norm)

    n = len(toks_vis)
    candidatos_por_token: Dict[str, List[str]] = {}
    spans = detectar_ngramas(toks_norm, idx_frase)
    ocupado = [False]*n
    preasignados: Dict[int, List[str]] = {}

    # 1) N-gramas primero
    for (i0, i1, frase_norm, ids) in spans:
        vtok = vecs_tok[i0]
        pool = [pid for pid in ids if _token_pasa_blocklist(etiquetas_norm[pid], dominio)]
        topk = topk_candidatos_para_token(" ".join(toks_vis[i0:i1]), vtok, vec_cls,
                                          pool, emb_etq, etiquetas_norm, etiquetas_txt,
                                          tokens_oracion_set, bigramas_oracion, dominio,
                                          K=k_por_token)

        preasignados[i0] = topk
        for k in range(i0, i1):
            ocupado[k] = True
            if k != i0:
                candidatos_por_token[toks_vis[k]] = [] if OMITIR_STOPWORDS_EN_SALIDA else [""]

    # 2) Tokens simples
    for i, (tok_vis, tok_norm) in enumerate(zip(toks_vis, toks_norm)):
        if i in preasignados:
            candidatos_por_token[tok_vis] = preasignados[i]
            continue

        # stopwords / ocupados
        if ocupado[i] or es_stop_or_punt(tok_vis):
            if not OMITIR_STOPWORDS_EN_SALIDA:
                candidatos_por_token[tok_vis] = [""]
            continue

        # Pool robusto
        if len(tok_norm) <= TOKEN_CORTO_EXACTO:
            pool = []
            if tok_norm in idx_token:
                pool += list(idx_token[tok_norm])
            for s in SINONIMOS_BASE.get(tok_norm, set()):
                if s in idx_token:
                    pool += list(idx_token[s])
            pool = list({pid for pid in pool if _token_pasa_blocklist(etiquetas_norm[pid], dominio)})[:m_pool]
        else:
            pool = pool_lexico(tok_norm, idx_token, etiquetas_norm, M=m_pool, dominio=dominio)

        if not pool:
            if not OMITIR_STOPWORDS_EN_SALIDA:
                candidatos_por_token[tok_vis] = [""]
            continue

        topk = topk_candidatos_para_token(tok_vis, vecs_tok[i], vec_cls,
                                          pool, emb_etq, etiquetas_norm, etiquetas_txt,
                                          tokens_oracion_set, bigramas_oracion, dominio,
                                          K=k_por_token)

        if topk == [""] and OMITIR_STOPWORDS_EN_SALIDA:
            # Si no pasa umbral y omitimos vacíos, NO incluimos el token
            continue
        candidatos_por_token[tok_vis] = topk

    return toks_vis, candidatos_por_token

```

```

preasignados[i0] = topk
for k in range(i0, i1):
    ocupado[k] = True
    if k != i0:
        candidatos_por_token[toks_vis[k]] = [] if OMITIR_STOPWORDS_EN_SALIDA else [""]

# 2) Tokens simples
for i, (tok_vis, tok_norm) in enumerate(zip(toks_vis, toks_norm)):
    if i in preasignados:
        candidatos_por_token[tok_vis] = preasignados[i]
        continue

    # stopwords / ocupados
    if ocupado[i] or es_stop_or_punt(tok_vis):
        if not OMITIR_STOPWORDS_EN_SALIDA:
            candidatos_por_token[tok_vis] = [""]
        continue

    # Pool robusto
    if len(tok_norm) <= TOKEN_CORTO_EXACTO:
        pool = []
        if tok_norm in idx_token:
            pool += list(idx_token[tok_norm])
        for s in SINONIMOS_BASE.get(tok_norm, set()):
            if s in idx_token:
                pool += list(idx_token[s])
        pool = list({pid for pid in pool if _token_pasa_blocklist(etiquetas_norm[pid], dominio)})[:m_pool]
    else:
        pool = pool_lexico(tok_norm, idx_token, etiquetas_norm, M=m_pool, dominio=dominio)

    if not pool:
        if not OMITIR_STOPWORDS_EN_SALIDA:
            candidatos_por_token[tok_vis] = [""]
        continue

    topk = topk_candidatos_para_token(tok_vis, vecs_tok[i], vec_cls,
                                      pool, emb_etq, etiquetas_norm, etiquetas_txt,
                                      tokens_oracion_set, bigramas_oracion, dominio,
                                      K=k_por_token)

    if topk == [""] and OMITIR_STOPWORDS_EN_SALIDA:
        # Si no pasa umbral y omitimos vacíos, NO incluimos el token
        continue
    candidatos_por_token[tok_vis] = topk

return toks_vis, candidatos_por_token

```

Explicación Línea por Línea:

Línea	Explicación
def topk_candidatos_para_token(...):	Función que rankea los candidatos para un token.
if not pool_ids:	Si no hay candidatos en el pool, retorna [""].
scores = []	Inicializa lista para almacenar puntuaciones.
for pid in pool_ids:	Itera sobre cada ID de pictograma candidato.
en = etiquetas_norm[pid]	Obtiene la etiqueta normalizada del pictograma.
if not _token_pasa_blocklist(en, dominio):	Si no pasa el filtro de contenido, lo omite.
continue	Salta al siguiente candidato.
ve = emb_etq[pid]	Obtiene el embedding de la etiqueta del pictograma.
sim_emb = float(np.dot(vec_token, ve))	Calcula similitud de embedding entre el token y la etiqueta.
sim_ctx = float(np.dot(vec_oracion_cls, ve))	Calcula similitud con el contexto global de la oración.
lex = score_lexico(token, en)	Calcula puntuación léxica.
pen = penalty_pos(token, en)	Calcula penalización por categoría gramatical.
b_cooc = bonus_coocurrencia(...)	Calcula bono por co-ocurrencia.
b_bi = bonus_bigrama(...)	Calcula bono por bigrama.
b_dom = bonus_dominio(...)	Calcula bono por dominio.
s = combinar_score(...)	Combina todas las puntuaciones.
scores.append((s, pid))	Almacena puntuación e ID.
if not scores:	Si no hay candidatos con puntuación, retorna [""].
scores.sort(reverse=True, key=lambda x: x[0])	Ordena candidatos por puntuación descendente.

if scores[o][o] < UMBRAL_SCORE:	Si el mejor candidato no supera el umbral, retorna [""].
out = []	Lista para los mejores candidatos.
for _, pid in scores[:K]:	Itera sobre los K mejores.
out.append(f'{pid}-{etiquetas_txt[pid].replace(' ', '_')}.jpg')	Formatea el nombre del archivo.
return out or [""]	Retorna la lista o [""].
def construir_bigrama_set(...):	Construye bigramas a partir de tokens.
toks = [t for t in tokens_norm ...]	Filtra tokens que no son stopwords.
return set(zip(toks, toks[1:])) ...	Retorna conjuntos de bigramas.
def candidatos_para_oracion(...):	Función principal para una oración.
toks_vis = [t for t in ...]	Tokeniza la oración en tokens visibles.
toks_base = lematiza_tokens(toks_vis) ...	Lematiza los tokens (si spaCy está disponible).
toks_norm = [norm_text(t) ...]	Normaliza los tokens.
vec_cls = embed_sentence_cls(oracion_str)	Obtiene embedding CLS de la oración.
vecs_tok = embed_tokens_en_context(...)	Obtiene embeddings contextuales para cada token.
tokens_oracion_set = set(...)	Crea conjunto de tokens de la oración (sin stopwords).
bigramas_oracion = construir_bigrama_set(toks_norm)	Construye bigramas de la oración.
n = len(toks_vis)	Número de tokens visibles.
candidatos_por_token: Dict[str, List[str]] = {}	Inicializa diccionario de salida.
spans = detectar_ngramas(toks_norm, idx_frase)	Detecta n-gramas en la oración.
ocupado = [False]*n	Array para marcar tokens cubiertos por n-gramas.
preasignados: Dict[int, List[str]] = {}	Diccionario para candidatos preasignados por n-gramas.
for (io, i1, frase_norm, ids) in spans:	Itera sobre los n-gramas detectados.
vtok = vecs_tok[io]	Toma el embedding del primer token del n-grama.

pool = [pid for pid in ids ...]	Filtra los IDs del n-grama por blocklist.
topk = topk_candidatos_para_token(...)	Obtiene los mejores candidatos para el n-grama.
preasignados[i0] = topk	Asigna los candidatos al primer token del n-grama.
for k in range(i0, i1):	Marca todos los tokens del n-grama como ocupados.
ocupado[k] = True	Marca token como ocupado.
if k != i0:	Para los tokens que no son el primero del n-grama.
candidatos_por_token[toks_vis[k]] = ...	Asigna lista vacía (o [""]) si no se omiten stopwords).
for i, (tok_vis, tok_norm) in enumerate(...):	Itera sobre cada token visible y normalizado.
if i in preasignados:	Si el token tiene candidatos preasignados (por n-grama).
candidatos_por_token[tok_vis] = preasignados[i]	Asigna los candidatos preasignados.
continue	Salta al siguiente token.
if ocupado[i] or es_stop_or_punt(tok_vis):	Si el token está ocupado o es stopword/puntuación.
if not OMITIR_STOPWORDS_EN_SALIDA:	Si no se omiten stopwords en salida.
candidatos_por_token[tok_vis] = [""]	Asigna candidato vacío.
continue	Salta al siguiente token.
if len(tok_norm) <= TOKEN_CORTO_EXACTO:	Si el token es corto, usa búsqueda exacta.
pool = []	Inicializa pool para token corto.
if tok_norm in idx_token:	Si el token está en el índice.
pool += list(idx_token[tok_norm])	Agrega candidatos exactos.
for s in SINONIMOS_BASE.get(tok_norm, set()):	Itera sobre sinónimos.
if s in idx_token:	Si el sinónimo está en el índice.
pool += list(idx_token[s])	Agrega candidatos del sinónimo.

pool = list({pid for pid in pool ...})[:m_pool]	Filtra por blacklist y limita a m_pool.
else:	Para tokens largos, usa pool léxico robusto.
pool = pool_lexico(...)	Obtiene pool de candidatos.
if not pool:	Si no hay candidatos.
if not OMITIR_STOPWORDS_EN_SALIDA:	Si no se omiten stopwords.
candidatos_por_token[tok_vis] = [""]	Asigna candidato vacío.
continue	Salta al siguiente token.
topk = topk_candidatos_para_token(...)	Rankea los candidatos para el token.
if topk == [""] and OMITIR_STOPWORDS_EN_SALIDA:	Si no hay candidatos y se omiten stopwords.
continue	No incluye el token en la salida.
candidatos_por_token[tok_vis] = topk	Asigna los candidatos al token.
return toks_vis, candidatos_por_token	Retorna los tokens visibles y el diccionario de candidato

Este bloque integra todo el proceso para una oración: desde la detección de n-gramas, la generación de pools de candidatos para cada token, hasta el ranking compuesto y la asignación final de pictogramas.

Entradas:

- token: Token individual a procesar
- vec_token: Embedding contextual del token
- vec_oracion_cls: Embedding CLS de la oración completa
- pool_ids: Lista de IDs de pictogramas candidatos
- Estructuras de datos de pictogramas (etiquetas, embeddings, índices)
- Información contextual de la oración (tokens, bigramas)
- oracion_str: Oración completa a procesar
- Parámetros de configuración (K, M, dominio)

Salidas:

- topk_candidatos_para_token: Lista de nombres de archivo de pictogramas seleccionados
- construir_bigrama_set: Conjunto de bigramas extraídos de tokens
- candidatos_para_oracion: Tupla con tokens visibles y diccionario de candidatos por token

Bloque 9: Construir Candidatos para un XML y Guardar a JSON (con Progreso)

Propósito del Bloque: Procesar un archivo XML que contiene oraciones, generar para cada oración un conjunto de candidatos de pictogramas (usando las funciones definidas en los bloques anteriores) y guardar los resultados en un archivo JSON con metadatos.

```
# @title Construir candidatos para un XML y guardar a JSON (con progreso)
def _inferir_dominio_desde_path(path_xml: str) -> str|None:
    p = path_xml.lower()
    if "salud" in p: return "salud"
    if "educacion" in p or "educación" in p: return "educacion"
    if "cultura" in p: return "cultura"
    return None

from tqdm.auto import tqdm

def construir_candidatos_json(path_xml: str, dir_pictos: str, path_out_json: str,
                             k_por_token=K_POR_TOKEN, m_pool=M_POOL, dominio: str|None=None):
    oraciones = leer_oraciones_xml(path_xml)
    dom = dominio or _inferir_dominio_desde_path(path_xml)

    salida = {"oraciones": [], "metadata": {
        "modelo": MODEL_NAME,
        "k_por_token": k_por_token,
        "m_pool": m_pool,
        "umbral_score": UMBRAL_SCORE,
        "omitir_stopwords_en_salida": OMITIR_STOPWORDS_EN_SALIDA,
        "dominio": dom
    }}

    for S in tqdm(oraciones, desc=f"Procesando {os.path.basename(path_xml)}"):
        toks, cand = candidatos_para_oracion(
            S, etiquetas_txt, etiquetas_norm, idx_token, idx_frase, emb_etq,
            dominio=dom, k_por_token=k_por_token, m_pool=m_pool
        )
        # Si OMITIR_STOPWORDS_EN_SALIDA=True, cand puede no incluir algunas claves; mantenemos tokens "visibles" por si quieres auditarlos
        salida["oraciones"].append({
            "oracion": S,
            "tokens": toks,
            "candidatos": cand
        })

    with open(path_out_json, "w", encoding="utf-8") as f:
        json.dump(salida, f, ensure_ascii=False, indent=2)
    print(f"Guardado: {path_out_json} ({len(salida['oraciones'])} oraciones)")
```

Explicación Línea por Línea:

Línea	Explicación
def _inferir_dominio_desde_path(path_xml: str) -> str None:	Función auxiliar para inferir el dominio a partir de la ruta del XML.
p = path_xml.lower()	Convierte la ruta a minúsculas para comparar sin distinción de caso.
if "salud" in p: return "salud"	Si la ruta contiene "salud", retorna "salud".
if "educacion" in p or "educación" in p: return "educacion"	Si contiene "educacion" o "educación", retorna "educacion".

if "cultura" in p: return "cultura"	Si contiene "cultura", retorna "cultura".
return None	Si no coincide, retorna None.
from tqdm.auto import tqdm	Importa la barra de progreso.
def construir_candidatos_json(...):	Función principal para construir el JSON de candidatos.
oraciones = leer_oraciones_xml(path_xml)	Lee las oraciones del archivo XML.
dom = dominio or _inferir_dominio_desde_path(path_xml)	Determina el dominio (si no se proporciona, lo infiere).
salida = {"oraciones": [], "metadata": { ... }}	Inicializa la estructura de salida con metadatos.
for S in tqdm(oraciones, desc=f"Procesando ..."):	Itera cada oración con una barra de progreso.
toks, cand = candidatos_para_oracion(...)	Procesa la oración para obtener tokens y candidatos.
salida["oraciones"].append({ ... })	Agrega los resultados de la oración a la salida.
with open(path_out_json, "w", encoding="utf-8") as f:	Abre el archivo de salida en modo escritura.
json.dump(salida, f, ensure_ascii=False, indent=2)	Escribe el JSON con formato legible.
print(f"Guardado: ...")	Mensaje de confirmación.

Flujo detallado:

1. **Inferir dominio:** A partir de la ruta del XML, se intenta inferir el dominio (salud, educación, cultura) si no se proporciona uno.
2. **Leer oraciones:** Se lee el archivo XML y se extraen las oraciones.
3. **Inicializar estructura de salida:** Se crea un diccionario que contendrá una lista de oraciones y metadatos.
4. **Procesar cada oración:** Para cada oración, se utiliza la función `candidatos para oración` (definida en el bloque anterior) para obtener los tokens visibles y los candidatos de pictogramas para cada token.
5. **Almacenar resultados:** Por cada oración, se guarda el texto original, los tokens visibles y el diccionario de candidatos.
6. **Guardar JSON:** Se escribe la estructura de datos en un archivo JSON con codificación UTF-8 y formato indentado.

7. **Mensaje de confirmación:** Se imprime la ruta del archivo guardado y el número de oraciones procesadas.

Este bloque actúa como el orquestador principal que utiliza todas las funciones definidas previamente para procesar un conjunto de oraciones y generar un archivo JSON con los pictogramas candidatos para cada token de cada oración.

Entradas del Bloque:

- path_xml: Ruta al archivo XML de entrada con las oraciones.
- dir_pictos: Ruta al directorio de pictogramas (aunque en este caso no se usa directamente, porque ya se cargaron previamente).
- path_out_json: Ruta de salida para el archivo JSON.
- k_por_token: Número de candidatos a retener por token (por defecto K_POR_TOKEN).
- m_pool: Tamaño del pool léxico inicial (por defecto M_POOL).
- dominio: Dominio específico (salud, educación, cultura) o None para inferirlo.

Salidas del Bloque:

- Un archivo JSON en path_out_json con la estructura:

```
{
  "oraciones": [
    {
      "oracion": "texto de la oración",
      "tokens": [lista de tokens visibles],
      "candidatos": {diccionario de token -> lista de candidatos}
    },
    ...
  ],
  "metadata": { ... }
}
```

Bloque 10: Ejecutar Pipeline v3 para las 3 Categorías

Propósito del Bloque: Ejecutar el proceso completo de generación de candidatos de pictogramas para los tres dominios definidos (salud, educación y cultura), utilizando las rutas de entrada y salida configuradas previamente.

```
# @title Ejecutar pipeline v3 para las 3 categorías
construir_candidatos_json(RUTA_XML_SALUD,      RUTA_PICTOS, OUT_SALUD)
construir_candidatos_json(RUTA_XML_EDUCACION, RUTA_PICTOS, OUT_EDUCACION)
construir_candidatos_json(RUTA_XML_CULTURA,  RUTA_PICTOS, OUT_CULTURA)
```

Explicación Línea por Línea:

Línea	Explicación
construir_candidatos_json(RUTA_XML_SALUD, RUTA_PICTOS, OUT_SALUD)	Ejecuta el pipeline para el dominio de salud: lee el

	XML de salud, procesa cada oración, genera los candidatos y guarda el resultado en el archivo JSON de salida para salud.
construir_candidatos_json(RUTA_XML_EDUCACION, RUTA_PICTOS, OUT_EDUCACION)	Ejecuta el pipeline para el dominio de educación: lee el XML de educación, procesa cada oración, genera los candidatos y guarda el resultado en el archivo JSON de salida para educación.
construir_candidatos_json(RUTA_XML_CULTURA, RUTA_PICTOS, OUT_CULTURA)	Ejecuta el pipeline para el dominio de cultura: lee el XML de cultura, procesa cada oración, genera los candidatos y guarda el resultado en el archivo JSON de salida para cultura.

Flujo de Ejecución Completo:

1. **Inferencia de Dominio:** Cada llamada automáticamente detecta el dominio basado en el nombre del archivo XML
2. **Procesamiento por Lotes:** Cada oración se procesa individualmente con barra de progreso
3. **Aplicación de Configuraciones Específicas:** Cada dominio aplica sus propias listas de boost y blacklist
4. **Generación Estructurada:** Los resultados se organizan en formato JSON con metadatos completos
5. **Persistencia:** Los archivos finales se guardan en las rutas especificadas

Entradas:

- RUTA_XML_SALUD: Archivo XML con oraciones del dominio de salud
- RUTA_XML_EDUCACION: Archivo XML con oraciones del dominio de educación
- RUTA_XML_CULTURA: Archivo XML con oraciones del dominio de cultura
- RUTA_PICTOS: Directorio base de pictogramas (ya indexado previamente)
- Rutas de salida preconfiguradas (OUT_SALUD, OUT_EDUCACION, OUT_CULTURA)

Salidas:

- OUT_SALUD: Archivo JSON con candidatos para oraciones de salud

- OUT_EDUCACION: Archivo JSON con candidatos para oraciones de educación
- OUT_CULTURA: Archivo JSON con candidatos para oraciones de cultura
- Mensajes de progreso y confirmación en consola para cada dominio

Glosario y Referencias

Términos Técnicos

- **Embedding:** Representación vectorial de texto
- **BERT:** Bidirectional Encoder Representations from Transformers
- **Token:** Unidad básica de texto procesado
- **Lematización:** Reducción de palabras a su forma base
- **N-grama:** Secuencia de N tokens consecutivos
- **Score de Similitud:** Medida numérica de relación semántica

Referencias

- [Documentación de Transformers](#)
- [Modelo BETO en español](#)
- [spaCy para español](#)
- [RapidFuzz documentación](#)

Estructuras de Datos Clave

- emb_etq: Diccionario de embeddings de pictogramas
- idx_token: Índice invertido token a pictogramas
- candidatos_por_token: Resultados finales por token

Este manual técnico proporciona la documentación completa para entender, ejecutar y mantener el sistema de recomendación de pictogramas.